

Lab - Statistics for Genomics

Introduction to R

Andrew Jaffe

TA

Learning Objectives

- Reading data into R
- Recoding and manipulating data
- Writing R functions and using add-on packages
- Making exploratory plots

Why R?

- Powerful and flexible
- Free (open source)
- Extensive add-on software
- Designed for statistical computing
- High level language
- Very widely used in genomics (Bioconductor)

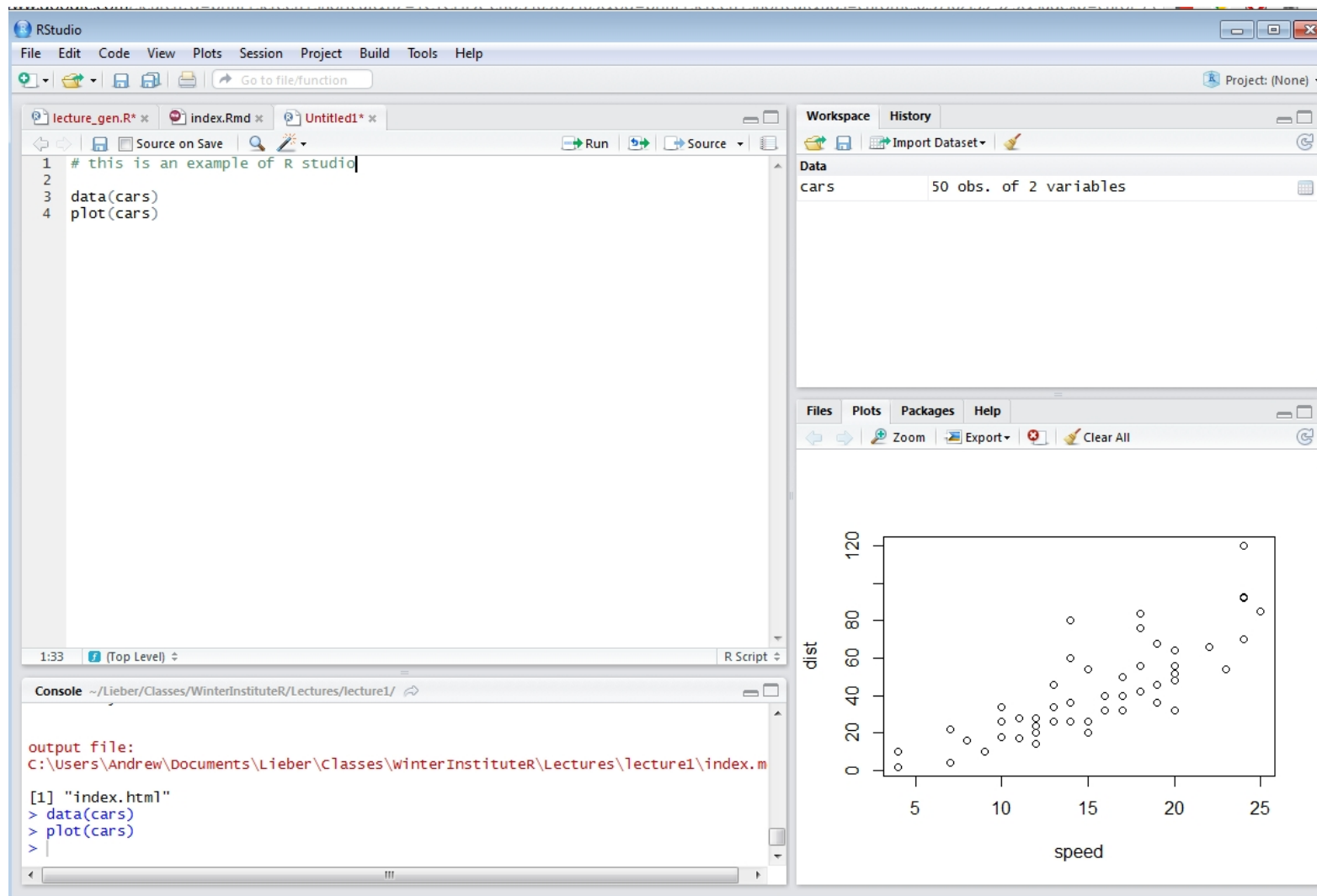
Installing R

Install the latest version from: <http://cran.r-project.org/>

Note that you must manually update R, often at your own peril...

R Studio

- Integrated Development Environment (IDE) for R
 - Syntax highlighting, code completion, and smart indentation
 - Execute R code directly from the source editor
 - Easily manage multiple working directories using projects
 - Workspace browser and data viewer
 - Plot history, zooming, and flexible image and PDF export
 - Integrated R help and documentation
 - Searchable command history
- <http://www.rstudio.com/>



Working with R

- The R Console "interprets" whatever you type
 - Calculator
 - Creating variables
 - Applying functions
- "Analysis" Script + Interactive Exploration
 - Static copy of what you did
 - Try things out interactively, then add to your script
- R revolves around 'functions'
 - Commands that take input, performs computations, and returns results
 - Many come with R, but people write external functions you can download and use

Getting Started

- You should have the latest version of R installed (R 2.15.2 as of 12/31/12)!
- Open R Studio
- Files --> New --> R Script
- Save the blank R script as "lab1.R" in a directory of your choosing
- Add a comment header

Commenting in Scripts

Add a comment header to lab1.R : '#' is the comment symbol

```
#####  
# Title: Demo R Script  
# Author: Andrew Jaffe  
# Date: 1/7/2013  
# Purpose: Demonstrate comments in R  
#####  
  
# this is a comment, nothing to the right of it gets read  
  
# this # is still a comment - you can use many #'s as you want  
  
# sometimes you have a really long comment, like explaining what you  
#   are doing for a step in analysis. Take it to a second line
```

Useful R Studio Shortcuts

- 'Ctrl+1' takes you to the script page
- 'Ctrl+2' takes you to the console
- 'Ctrl + Enter' ('Cmd + Enter' on OS X) in your script evaluates that line of code

Data Input

Postmortem Brain Expression Dataset (Colantuoni 2011) from the DLPFC brain region in 269 subjects

- Download gene expression data: <http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE30272>
- Supplementary file (at the bottom):

"GSE30272_RGna.n269.o30176.log2ratio.loess.MADout.KNNimp.txt.gz"

- Unzip it and place the text file in your working directory
- Also get the annotation file: http://rafalab.jhsph.edu/688/R/annotation_n269.zip

Data Input

```
> dat = read.table("GSE30272_RGna.n269.o30176.log2ratio.loess.MADout.KNNimp.txt",  
+   header = TRUE, as.is = TRUE, sep = "\t", row.names = 1)  
> dim(dat)
```

```
[1] 30176 269
```

```
> dat[1:5, 1:5]
```

	HB_18_34	HB_22_35	HB_16_29	HB_17_91	HB_18_16
hCT000600	-0.2919	-0.06015	-0.27251	-0.43951	-0.6232
hCT000696	-0.2147	0.20541	-0.35320	-0.20986	-0.6733
hCT000684	0.1640	0.14871	-0.08211	-0.03027	-0.3942
hCT001080	0.3595	1.31613	0.47240	0.29985	-0.5491
hCT001176	-0.2583	-0.39569	-0.65895	-0.57300	-0.9397

```
> names(dat)[1:10]
```

```
[1] "HB_18_34" "HB_22_35" "HB_16_29" "HB_17_91" "HB_18_16" "HB_16_62"  
[7] "HB_18_35" "HB_20_84" "HB_20_80" "HB_17_26"
```

Functions

- R revolves around functions: denoted by "`()`"
- Every function takes an input, defined by arguments, often provided by the user
- Many functions have default settings for these arguments
- If you know the name of a function, "`?[function name]`" or '`help([function name])`' will pop up the help menu
- '`example([function name])`' shows you how it is used

Working Directory

- R looks for files on your computer relative to the "working" directory
- It's always safer to set the working directory at the beginning of your script
- Example of help file

```
> ## get the working directory  
> getwd()
```

```
[1] "F:/Hopkins/Lieber/Classes/Genomics2013/introR"
```

```
> ### set the working directory  
> setwd("../")  
> getwd()
```

```
[1] "F:/Hopkins/Lieber/Classes/Genomics2013"
```

Working Directory

- Setting the directory can sometimes be finicky
 - Windows: Default directory structure involves single backslashes ("\"), but R interprets these as "escape" characters. So you must replace the backslash with forward slashed ("/") or two backslashes ("\\")
 - Mac/Linux: Default is forward slashes, so you are okay
- Regular directory structure shortcuts apply
 - ".." goes up one level
 - "./" is the current directory
 - "~" is your home directory

Working Directory

Try some directory navigation:

```
> dir("./") # shows directory contents
```

```
[1] "annotation_n269.txt"
[2] "annotation_n269.zip"
[3] "assets"
[4] "cache"
[5] "GPL4611.annot"
[6] "GSE30272_RGna.n269.o30176.log2ratio.loess.MADout.KNNimp.txt"
[7] "index.html"
[8] "index.md"
[9] "index.Rmd"
[10] "Lecture 1.pdf"
[11] "lecture1_code.R"
[12] "lecture1_test.html"
[13] "libraries"
[14] "pics"
```

```
> dir("../")
```

```
[1] "introR"
```

```
> head(dir("F:/Hopkins/Lieber"))
```


Working Directory

- Set your working directory to the same location where you saved the blank R script: "lab1.R"
- Confirm the directory contains "lab1.R"

R variables

- You can create variables from within the R environment and from files on your computer
- R uses "=" or "<-" to assign values to a variable name
- Variable names are case-sensitive, i.e. X and x are different

```
> x = 2  
> x
```

```
[1] 2
```

```
> x * 4
```

```
[1] 8
```

```
> x + 2
```

```
[1] 4
```

R variables

- Each variable has a 'class' associated with it

```
> class(x)
```

```
[1] "numeric"
```

```
> y = "hello world!"  
> print(y)
```

```
[1] "hello world!"
```

```
> class(y)
```

```
[1] "character"
```

The 'combine' function

The function 'c()' collects/combines/joins single R objects into a vector of R objects. It is mostly used for creating vectors of numbers, character strings, and other data types.

```
> x <- c(1, 4, 6, 8)
> x
```

```
[1] 1 4 6 8
```

```
> str(x)
```

```
num [1:4] 1 4 6 8
```

R variables

`length()`: Get or set the length of vectors (including lists) and factors, and of any other R object for which a method has been defined.

```
> length(x)
```

```
[1] 4
```

```
> y
```

```
[1] "hello world!"
```

```
> length(y)
```

```
[1] 1
```

Data Input

- 'Reading in' data is the first step of any project/analysis
- R can read almost any file format, especially via add-on packages
- We are going to focus on simple delimited files first
 - tab delimited (e.g. '.txt') - `read.delim()`
 - comma separated (e.g. '.csv') - `read.csv()`
 - Microsoft excel (e.g. '.xlsx') - `xlsx::read.xlsx`

Data Input

```
> map = read.delim("annotation_n269.txt", header = TRUE, as.is = TRUE)
> rownames(map) = map$OligoID
> map[1:2, ]
```

	ID	SeqId		Seq			
hCT000600	600	20302					
hCT000696	696	5526					
hCT000600				TGGGTGTGCCCCGACTTAAGGAGCTCATCAACATTTCCAAGAAGCCAAAGACTCCTTCGCTTACTGTCTT			
hCT000696				TACAACAGCACTTGTGACGACCGTGAGTTTCATGTGCCAGAACCGCCAGTGCATCCCCAAGCACTTCGTGT			
	Type	GenomicMatches	mRNAMatches	TM	GC	EntrezGeneID	Gene
hCT000600	hCT	1	1	81.81	0.49	5430	POLR2A
hCT000696	hCT	1	1	84.15	0.54	4035	LRP1
							Description
hCT000600							polymerase (RNA) II (DNA directed) polypeptide A, 220kDa
hCT000696							low density lipoprotein-related protein 1 (alpha-2-macroglobulin receptor)
	GeneIDs	GenePos	Mapped				
hCT000600		chr17:7328573-7358655,	RSQ				
hCT000696		chr12:55808548-55893392,	RSQ				
	ProbePos	ProbeCytoband	Accession	LocusLinkID			
hCT000600	+chr17:7352409-7352478,	17	NM_000937,	5430			
hCT000696	+chr12:55875710-55875779,	12	NM_002332,	4035			
	Intensity	bestMode					
hCT000600	8.728	75					
hCT000696	8.411	57					
							Link
hCT000600							http://www.ncbi.nlm.nih.gov/entrez/dispomim.cgi?id=180660
hCT000696							http://www.ncbi.nlm.nih.gov/entrez/dispomim.cgi?id=107770
	GeneStart1	GeneEnd1	chr1	snpInProbe	probeStart	ProbeEnd	
hCT000600	7328573	7358655	chr17	0	7352409	7352478	
hCT000696	55808548	55893392	chr12	1	55875710	55875779	
	HapSnpInProbe	NCBImaplocation	estMatch	geneName	geneNameId		

Data Input

There is a 'wrapper' function for reading CSV files:

```
read.csv
```

```
## function (file, header = TRUE, sep = ",", quote = "\"", dec = ".",  
##     fill = TRUE, comment.char = "", ...)  
## read.table(file = file, header = header, sep = sep, quote = quote,  
##     dec = dec, fill = fill, comment.char = comment.char, ...)  
## <bytecode: 0x000000002d043958>  
## <environment: namespace:utils>
```

Note: the '...' designates arguments that are passed to read.table()

Head and Tail

- `head()` shows the first 6 (default) elements of an R object
- `tail()` shows the last 6 (default) elements of an R object

```
> z = 1:100  
> head(z)
```

```
[1] 1 2 3 4 5 6
```

```
> tail(z)
```

```
[1] 95 96 97 98 99 100
```

Data Input

The `read.table()` function returns a 'data.frame'

```
> class(map)
```

```
[1] "data.frame"
```

```
> str(map)
```

```
'data.frame': 30176 obs. of 44 variables:
 $ ID      : int  600 696 684 1080 1176 1260 1464 1452 1656 1752 ...
 $ SeqId   : int  20302 5526 26038 39363 5155 14379 42944 28547 37026 8019 ...
 $ Seq     : chr   "TGGGTGTGCCCCGACTTAAGGAGCTCATCAACATTTCCAAGAAGCCAAAGACTCCTTCGCTTACTGTCTT" "TACA"
 $ Type    : chr   "hCT" "hCT" "hCT" "hCT" ...
 $ GenomicMatches : int  1 1 1 1 0 0 1 0 1 1 ...
 $ mRNAMatches  : int  1 1 1 1 1 1 1 1 1 1 ...
 $ TM       : num   81.8 84.2 85.9 83 87.7 ...
 $ GC       : num   0.49 0.54 0.59 0.51 0.63 0.43 0.23 0.4 0.29 0.23 ...
 $ EntrezGeneID : int  5430 4035 7094 4952 310 4952 254531 5925 79657 84455 ...
 $ Gene      : chr   "POLR2A" "LRP1" "TLN1" "OCRL" ...
 $ Description : chr   "polymerase (RNA) II (DNA directed) polypeptide A, 220kDa" "low density lipoproc
 $ GeneIDs    : chr   "" "" "" "" ...
 $ GenePos    : chr   "chr17:7328573-7358655," "chr12:55808548-55893392," "chr9:35687333-35722392," '
 $ Mapped     : chr   "RSQ" "RSQ" "RSQ" "RSQ" ...
 $ ProbePos   : chr   "+chr17:7352409-7352478," "+chr12:55875710-55875779," "-chr9:35696230-35696299,
 $ ProbeCytoband : chr   "17p13.1" "12q13.3" "9p13.3" "Xq25" ...
 $ Accession  : chr   "NM_000937," "NM_002332," "NM_006289," "NM_000276," ...
 $ LocusLinkID : int  5430 4035 7094 4952 310 4952 254531 5925 79657 84455 ...
 $ Intensity  : num   8.73 8.41 8 7.79 9.12 ...
```

Data Classes:

One dimensional classes:

- Character: strings or individual characters, quoted
- Numeric: any real number(s)
- Integer: any integer(s)/whole numbers
- Factor: categorical/qualitative variables that can be ordinal
- Logical: variables composed of TRUE or FALSE

Two dimensional classes:

- Data frame: traditional 'excel' spreadsheets
 - Each column can have a different class, from above
 - There are column names and sometimes row names
- Matrix: two-dimensional data, composed of rows and columns. Unlike data frames, the entire matrix is composed of one R class.

Matrices

```
> n = 1:9 # sequence from first number to second number incrementing by 1  
> n
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
> mat = matrix(n, nr = 3)  
> mat
```

```
      [,1] [,2] [,3]  
[1,]    1    4    7  
[2,]    2    5    8  
[3,]    3    6    9
```

Matrix and Data frame Attributes

- `nrow()` displays the number of rows of a matrix or data frame
- `ncol()` displays the number of columns
- `dim()` displays a vector of length 2: # rows, # columns
- `colnames()` displays the column names (if any) and `rownames()` displays the row names (if any)

Data Selection

Brackets are used to select and/or subset data in R

```
> x1 = 10:20  
> x1
```

```
[1] 10 11 12 13 14 15 16 17 18 19 20
```

```
> length(x1)
```

```
[1] 11
```

Data Selection

```
> x1[1] # selecting first element
```

```
[1] 10
```

```
> x1[3:4] # selecting third and fourth elements
```

```
[1] 12 13
```

```
> x1[c(1, 5, 7)] # selecting first, fifth, and seventh elements
```

```
[1] 10 14 16
```

Data Selection

Matrices have two "slots": rows and columns

```
> mat[1, 1] # individual entry: row 1, column 1
```

```
[1] 1
```

```
> mat[1, ] # first row
```

```
[1] 1 4 7
```

```
> mat[, 1] # first columns
```

```
[1] 1 2 3
```


Data Selection

Note that the class of the returned object is no longer a matrix

```
> class(mat[1, ])
```

```
[1] "integer"
```

```
> class(mat[, 1])
```

```
[1] "integer"
```

Data Selection

Data frames have special ways to select data, specifically by a "\$" and the column name

```
> head(names(map))
```

```
[1] "ID"          "SeqId"       "Seq"         "Type"
[5] "GenomicMatches" "mRNAMatches"
```

```
> head(map$EntrezGeneID)
```

```
[1] 5430 4035 7094 4952 310 4952
```

```
> head(map$GenePos)
```

```
[1] "chr17:7328573-7358655," "chr12:55808548-55893392,"
[3] "chr9:35687333-35722392," "chrX:128501932-128554211,"
[5] "chr10:74805194-74843847," "chrX:128501932-128554211,"
```

Data Selection

You can also subset data frames like matrices, using row and column indices, but using column names is generally safer and more reproducible.

```
> head(map[, 2])
```

```
[1] 20302 5526 26038 39363 5155 14379
```

You can also use the bracket notation, but specify the name(s) in quotes if you want more than 1 column. This allows you to subset rows and columns at the same time

```
> map[1:3, c("GenePos", "EntrezGeneID")]
```

	GenePos	EntrezGeneID
hCT000600	chr17:7328573-7358655,	5430
hCT000696	chr12:55808548-55893392,	4035
hCT000684	chr9:35687333-35722392,	7094

Unique entries

`unique()`: `unique` returns a vector, data frame or array like `x` but with duplicate elements/rows removed.

```
> x = c(1, 2, 3, 4, 4, 5, 4, 5)
> unique(x)
```

```
[1] 1 2 3 4 5
```

Note that this does NOT sort entries - it just returns the unique entries in the order it encounters them from first to last:

```
> unique(c(5, 1, 2, 3, 4, 4, 5, 4, 5))
```

```
[1] 5 1 2 3 4
```

Tabulating

`table()`: `table` uses the cross-classifying factors to build a contingency table of the counts at each combination of factor levels.

```
> table(map$Type)
```

hCT	hCX	hHA	hHC	hHE	hHO	hHR
472	17	6475	15967	814	370	6061

You can tabulate across as many variables as you like, but anything past 2 typically gets hard to interpret

Changing Attributes, like names

Names are just an attribute of the data frame (recall `str`) that you can change to any valid character name

Valid character names are case-sensitive, contain a-z, 0-9, underscores, and periods (but cannot start with a number).

For data frames, `colnames()` and `names()` return the same attribute.

```
> names(map) [1:5]
```

```
[1] "ID"          "SeqId"       "Seq"         "Type"
[5] "GenomicMatches"
```

```
> names(map)[1] = "ProbeID"
> names(map) [1:5]
```

```
[1] "ProbeID"      "SeqId"       "Seq"         "Type"
[5] "GenomicMatches"
```

Logical Statements

The logical class consists of either TRUE or FALSE

```
> z = c(TRUE, FALSE, TRUE, FALSE)
> class(z)
```

```
[1] "logical"
```

```
> sum(z)
```

```
[1] 2
```

Note that the logical class does NOT use quotes.

```
> z2 = c("TRUE", "FALSE", "TRUE", "FALSE")  
> class(z2)
```

```
[1] "character"
```

```
> sum(z2)
```

```
Error: invalid 'type' (character) of argument
```

```
> identical(z, z2)
```

```
[1] FALSE
```


Logical Statements

- This mirrors computer science/programming syntax:
 - '==': equal to
 - '!=': not equal to (it is NOT '~' in R, e.g. SAS)
 - '>': greater than
 - '<': less than
 - '>=': greater than or equal to
 - '<=': less than or equal to

which()

?which: Give the TRUE indices of a logical object, allowing for array indices.

```
> which(map$GC > 0.6) [1:5]
```

```
[1] 5 86 112 250 282
```

```
> which(map$GC < 0.4) [1:5]
```

```
[1] 7 9 10 11 12
```

```
> which(map$geneName == "DRD2")
```

```
[1] 2396 29214
```

Missing Data

- In R, missing data is represented by the symbol NA (note that it is NOT a character, and therefore not in quotes)
- `is.na()` is a logical test for which variables are missing
- Many summarization functions do not the calculation you expect (e.g. they return NA) if there is ANY missing data, and these often have an argument `'na.rm=FALSE'`. Changing this to `'na.rm=TRUE'` will ignore the missing values in the calculation (i.e. `mean()`, `sd()`, `max()`, `sum()`)

This will help you with the homework: <http://www.statmethods.net/input/missingdata.html>

R classes

You can test whether an R object is a specific class. For example:

- `is.character(object)`
- `is.integer(object)`
- `is.numeric(object)`
- `is.factor(object)`
- `is.matrix(object)`
- `is.data.frame(object)`